



Fetch: Consumindo APIs Reais

Aula 14 · Bloco 4 — Assincronismo

Dados do mundo real entrando na sua página

Nesta aula

1. O que é uma **API**
2. `fetch` — e o padrão dos dois `await`
3. Os **dois tipos de erro** que você precisa tratar
4. Fetch + DOM: o app completo
5. APIs abertas para explorar

O que é uma API

Um endereço na internet que devolve dados em vez de páginas. Você acessa uma URL e recebe **JSON** — o da aula 06.

```
{ "cep": "01310-100", "logradouro": "Avenida Paulista",  
  "localidade": "São Paulo", "uf": "SP" }
```

💡 Abra viacep.com.br/ws/01310100/json/ no navegador agora. É isso: dados crus, sem página.

Praticamente todo app funciona assim

O front-end — JavaScript no navegador — **pede dados** a APIs e **desenha a tela**.

Instagram, iFood, banco: por baixo, é sempre um `fetch` buscando JSON e um `forEach` montando a lista.

Você já sabe fazer a segunda parte desde a aula 10.

fetch — o carteiro do JavaScript

```
const buscarCep = async (cep) => {  
  const resposta = await fetch(`https://viacep.com.br/ws/${cep}/json/`);  
  const dados = await resposta.json();    // extrair o JSON também demora!  
  console.log(dados);  
};
```

Os dois **await** são o padrão fixo:

1. **await fetch(url)** — espera a **resposta** do servidor chegar;
2. **await resposta.json()** — espera o **corpo** ser lido e virar objeto.

Daí em diante, **dados** é um objeto JS comum.

⚠ O `fetch` não rejeita em erro 404

Servidor respondeu **404** ou **500**?

Para o `fetch`, **a requisição foi um sucesso** —
ele entregou a resposta que veio.

Você precisa conferir na mão:

```
if (!resposta.ok) throw new Error(...)
```

Os dois tipos de erro

```
try {  
  const resposta = await fetch(url);  
  // TIPO 1 – o servidor respondeu, mas com erro:  
  if (!resposta.ok) throw new Error(`Erro ${resposta.status}`);  
  
  const dados = await resposta.json();  
  if (dados.erro) throw new Error("CEP não encontrado");  
  return dados;  
} catch (erro) {  
  // TIPO 2 – nem completou: sem internet, URL errada  
  console.log(`Falha: ${erro.message}`);  
}
```

throw new Error(...)

Lança um erro **de propósito**, capturado pelo `catch` mais próximo.

Isso centraliza todo o tratamento **num lugar só**, em vez de espalhar `if` por toda a função.

💡 Você vai reencontrar exatamente essa mecânica em Java: `throw`, `try`, `catch`. Mesma ideia, mesmo nome.

Fetch + DOM: o ciclo de feedback

```
const buscar = async () => {  
  status.textContent = "🔄 Buscando...";           // CARREGANDO  
  try {  
    const dados = await buscarCep(campo.value.trim());  
    status.textContent = "✅ Encontrado!";          // SUCESSO  
    resultado.textContent = dados.logradouro;  
  } catch (erro) {  
    status.textContent = `❌ ${erro.message}`;      // ERRO  
  }  
};
```

A validação da aula 11 entra antes do `try` — e o `return` corta ali mesmo.

O ciclo de feedback

carregando → **sucesso** ou **erro**

Todo aplicativo decente faz isso.

Sem o "carregando", o usuário clica de novo achando que não funcionou.

É requisito do projeto final.

APIs abertas para explorar

API	O que faz
ViaCEP	CEPs brasileiros
BrasilAPI	CEPs, feriados, bancos, DDDs
PokéAPI	dados de Pokémon
REST Countries	dados de países
Open-Meteo	previsão do tempo

💡 **Antes de programar, abra a URL no navegador** e estude o JSON. Algumas devolvem um **array** (`dados[0].nome`), outras um objeto com listas dentro.

Exercícios da aula

Na pasta `aula-14/` :

1. **Buscador de CEP** — teste os três cenários: válido, inexistente, mal formatado;
2. **Pokédex mínima** — nome, número e a imagem via `sprites.front_default`. Trate o 404;
3. **Feriados do ano** — BrasilAPI. Repare: a resposta é um **array**;
4. **Desafio** 🌶️ — `<select>` de meses filtrando os dados **já baixados**, sem buscar de novo.

Próxima aula

Aula 15 — Projeto Final

Tudo do curso, num aplicativo só —
seu, publicado, com o seu nome.